

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity and2_tb is
5 end and2_tb;
6
7 architecture behavior of and2_tb is
8     component and2 is
9     port (
10         a : in std_logic_vector(1 downto 0);
11         b : in std_logic_vector(1 downto 0);
12         z : out std_logic_vector(1 downto 0)
13     );
14     end component;
15
16     signal input_a, input_b :
17         std_logic_vector(1 downto 0);
18     signal output_z :
19         std_logic_vector(1 downto 0);
20
21 begin
22     UUT: and2 port map (a => input_a, b =>
23         input_b, z => output_z);
24
25     process
26     begin
27         input_a <= "00"; input_b <= "00"; wait
28         for 10 ns;
29         input_a <= "01"; input_b <= "01"; wait
30         for 10 ns;
31         input_a <= "10"; input_b <= "10"; wait
32         for 10 ns;
33         input_a <= "11"; input_b <= "11"; wait
34         for 10 ns;
35         wait;
36     end process;
37 end behavior;

```

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity and2 is
5     port (
6         a : in std_logic_vector(1 downto 0);
7         b : in std_logic_vector(1 downto 0);
8         z : out std_logic_vector(1 downto 0)
9     );
10 end and2;
11
12 architecture behavioral of and2 is
13 begin
14     z <= a and b;
15 end behavioral;
16

```

```

2 use ieee.std_logic_1164.all;
3
4 entity or2_tb is
5 end or2_tb;
6
7 architecture behavior of or2_tb is
8     component or2 is
9     port (
10         a : in std_logic_vector(1 downto 0);
11         b : in std_logic_vector(1 downto 0);
12         z : out std_logic_vector(1 downto 0)
13     );
14     end component;
15
16     signal input_a, input_b : std_logic_vector(1 downto 0);
17     signal output_z : std_logic_vector(1 downto 0);
18
19 begin
20     UUT: or2 port map (a => input_a, b => input_b, z => output_z);
21
22     process
23     begin
24         input_a <= "00"; input_b <= "00"; wait for 10 ns;
25         assert (output_z = "00") report "Test 1 failed" severity error;
26
27         input_a <= "00"; input_b <= "01"; wait for 10 ns;
28         assert (output_z = "01") report "Test 2 failed" severity error;
29
30         input_a <= "01"; input_b <= "00"; wait for 10 ns;
31         assert (output_z = "01") report "Test 3 failed" severity error;
32
33         input_a <= "01"; input_b <= "01"; wait for 10 ns;
34         assert (output_z = "01") report "Test 4 failed" severity error;
35
36         input_a <= "10"; input_b <= "00"; wait for 10 ns;
37         assert (output_z = "10") report "Test 5 failed" severity error;
38
39         input_a <= "10"; input_b <= "01"; wait for 10 ns;
40         assert (output_z = "11") report "Test 6 failed" severity error;
41
42         input_a <= "11"; input_b <= "00"; wait for 10 ns;
43         assert (output_z = "11") report "Test 7 failed" severity error;
44
45         input_a <= "11"; input_b <= "01"; wait for 10 ns;
46         assert (output_z = "11") report "Test 8 failed" severity error;
47
48         report "Testbench finished";
49         wait;
50     end process;
51 end behavior;

```

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity or2 is
5     port (
6         a : in std_logic_vector(1 downto 0);
7         b : in std_logic_vector(1 downto 0);
8         z : out std_logic_vector(1 downto 0)
9     );
10 end or2;
11
12 architecture behavioral of or2 is
13 begin
14     z <= a or b;
15 end behavioral;
16

```

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity half_adder_tb is
5 end half_adder_tb;
6
7 architecture behavior of half_adder_tb is
8     component half_adder is
9     port (
10         a : in std_logic;
11         b : in std_logic;
12         sum : out std_logic;
13         carry : out std_logic
14     );
15 end component;
16
17 signal input_a, input_b : std_logic;
18 signal output_sum, output_carry : std_logic;
19
20 begin
21     UUT: half_adder port map (a => input_a, b => input_b, sum => output_sum, carry
22 => output_carry);
23
24     process
25     begin
26         input_a <= '0'; input_b <= '0'; wait for 10 ns;
27         assert (output_sum = '0' and output_carry = '0') report "Test 1 failed"
28 severity error;
29
30         input_a <= '0'; input_b <= '1'; wait for 10 ns;
31         assert (output_sum = '1' and output_carry = '0') report "Test 2 failed"
32 severity error;
33
34         input_a <= '1'; input_b <= '0'; wait for 10 ns;
35         assert (output_sum = '1' and output_carry = '0') report "Test 3 failed"
36 severity error;
37
38         input_a <= '1'; input_b <= '1'; wait for 10 ns;
39         assert (output_sum = '0' and output_carry = '1') report "Test 4 failed"
40 severity error;
41
42         report "Testbench finished";
43         wait;
44     end process;
45 end behavior;
46

```

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity half_adder is
5     port (
6         a : in std_logic;
7         b : in std_logic;
8         sum : out std_logic;
9         carry : out std_logic
10    );
11 end half_adder;
12
13 architecture behavioral of half_adder is
14 begin
15     sum <= a xor b;
16     carry <= a and b;
17 end behavioral;
18

```

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity xor2_tb is
5 end xor2_tb;
6
7 architecture behavior of xor2_tb is
8     component xor2 is
9     port (
10         a : in  std_logic_vector(1 downto 0);
11         b : in  std_logic_vector(1 downto 0);
12         z : out std_logic_vector(1 downto 0)
13     );
14 end component;
15
16 signal input_a, input_b : std_logic_vector(1 downto 0);
17 signal output_z         : std_logic_vector(1 downto 0);
18
19 begin
20     UUT: xor2 port map (a => input_a, b => input_b, z => output_z);
21
22     process
23     begin
24         input_a <= "00"; input_b <= "00"; wait for 10 ns;
25         input_a <= "01"; input_b <= "01"; wait for 10 ns;
26         input_a <= "10"; input_b <= "10"; wait for 10 ns;
27         input_a <= "11"; input_b <= "11"; wait for 10 ns;
28         wait;
29     end process;
30 end behavior;
31

```

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity xor2 is
5     port (
6         a : in  std_logic_vector(1 downto 0);
7         b : in  std_logic_vector(1 downto 0);
8         z : out std_logic_vector(1 downto 0)
9     );
10 end xor2;
11
12 architecture behavioral of xor2 is
13 begin
14     z <= a xor b;
15 end behavioral;
16
17 |

```

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity full_adder_tb is
5 end full_adder_tb;
6
7 architecture behavior of full_adder_tb is
8     component full_adder is
9     port (
10         a : in  std_logic;
11         b : in  std_logic;
12         carryIn : in std_logic;
13         sum : out std_logic;
14         carryOut : out std_logic
15     );
16 end component;
17
18 signal input_a, input_b, input_cin : std_logic;
19 signal output_sum, output_cout : std_logic;
20
21 begin
22     UUT: full_adder port map (a => input_a, b => input_b, carryIn => input_cin,
23                             sum => output_sum, carryOut => output_cout);
24
25     process
26     begin
27         input_a <= '0'; input_b <= '0'; input_cin <= '0'; wait for 10 ns;
28         input_a <= '0'; input_b <= '0'; input_cin <= '1'; wait for 10 ns;
29         input_a <= '0'; input_b <= '1'; input_cin <= '0'; wait for 10 ns;
30         input_a <= '0'; input_b <= '1'; input_cin <= '1'; wait for 10 ns;
31         input_a <= '1'; input_b <= '0'; input_cin <= '0'; wait for 10 ns;
32         input_a <= '1'; input_b <= '0'; input_cin <= '1'; wait for 10 ns;
33         input_a <= '1'; input_b <= '1'; input_cin <= '0'; wait for 10 ns;
34         input_a <= '1'; input_b <= '1'; input_cin <= '1'; wait for 10 ns;
35         wait;
36     end process;
37 end behavior;

```

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity full_adder is
5     port (
6         a : in  std_logic;
7         b : in  std_logic;
8         carryIn : in std_logic;
9         sum : out std_logic;
10        carryOut : out std_logic
11    );
12 end full_adder;
13
14 architecture structural of full_adder is
15     component half_adder
16     port (
17         a : in  std_logic;
18         b : in  std_logic;
19         sum : out std_logic;
20         carry : out std_logic
21     );
22 end component;
23
24 signal ha1_sum, ha1_carry, ha2_sum, ha2_carry : std_logic;
25
26 begin
27     HA1: half_adder port map(a => a, b => b, sum => ha1_sum,
28                             carry => ha1_carry);
29     HA2: half_adder port map(a => ha1_sum, b => carryIn, sum =>
30                             sum, carry => ha2_carry);
31     carryOut <= ha1_carry or ha2_carry;
32 end structural;

```